



Concevez et analysez une base de données NoSQL

Sophana SEAN ————— Janvier 2026

Contexte du projet et importation des données

L'association NosCités analyse l'impact des locations courte durée et suite au crash de la base de données, nous restaurons une sauvegarde des listings parisiens de l'été 2024.

L'objectif est vérifier ces données et produire un rapport démontrant le retrait massif de logements du marché longue durée, afin de proposer des régulations renforcées aux pouvoirs publics.

Prérequis

MongoDB : base de données NoSQL

Compass : interface graphique

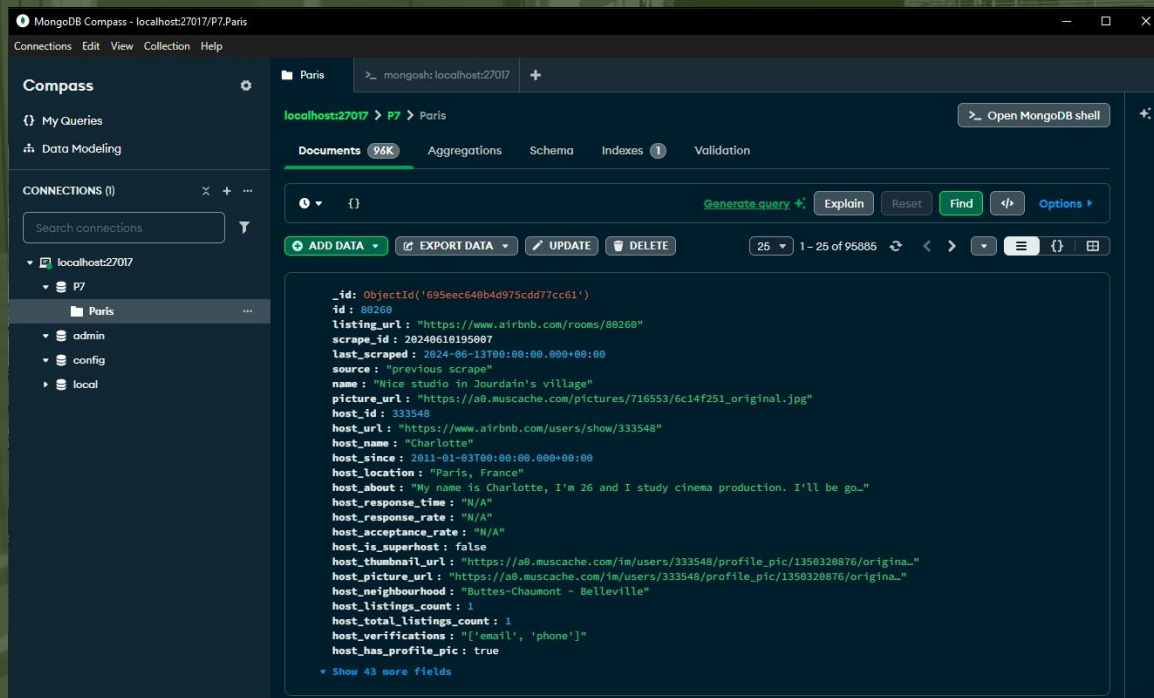
Base de données Mongo

Fichier CSV : listings_paris.csv

Nom de la BDD : P7

Collection : Paris

Données : 95885 lignes



01

Comprenez les bénéfices du NoSQL

Description d'un document avec le type des données

Premier document

compass : onglet "Documents"**id**: 80260**champs**: 74

The screenshot shows the Compass interface with the 'Documents' tab selected. The document is a JSON object representing an Airbnb listing. The fields include: `_id` (ObjectId), `id` (80260), `listing_url` (https://www.airbnb.com/rooms/80260), `scrape_id` (20240610195007), `last_scraped` (2024-06-13T00:00:00.000+00:00), `source` (previous scrape), `name` (Nice studio in Jourdain's village), `picture_url` (https://a0.muscache.com/pictures/716553/6c14f251_original.jpg), `host_id` (333548), and `host_url` (https://www.airbnb.com/users/show/333548).

```
{
  "_id": ObjectId('695eec640b4d975cdd77cc61'),
  "id": 80260,
  "listing_url": "https://www.airbnb.com/rooms/80260",
  "scrape_id": 20240610195007,
  "last_scraped": 2024-06-13T00:00:00.000+00:00,
  "source": "previous scrape",
  "name": "Nice studio in Jourdain's village",
  "picture_url": "https://a0.muscache.com/pictures/716553/6c14f251_original.jpg",
  "host_id": 333548,
  "host_url": "https://www.airbnb.com/users/show/333548"
}
```

Type de données

compass : onglet "Schema"**champs variés** : strings (name, host_about),
nombres (bedrooms), booléens
(host_is_superhost), arrays (amenities), dates

Données hétérogènes

Les logements peuvent avoir :

- des équipements différents
- des règles variables
- des informations parfois manquantes

Schéma flexible

- Ajout ou suppression de champs sans migration lourde
- Évolution facile du modèle de données dans le temps

Données semi-structurées

- Objets imbriqués (ex : hôte, localisation, disponibilité)
- Tableaux (équipements, règles...)

Scalabilité et performance

- Gros volumes de données
- Requêtes analytiques rapides sur des collections volumineuses

Requête 01.

Nombre de documents de la base de données

```
> db["Paris"].countDocuments()  
> db.Paris.countDocuments()
```

Résultat : 95885

```
>_MONGOSH
```

```
> use P7
```

```
< switched to db P7
```

```
> db["Paris"].countDocuments()
```

```
< 95885
```

Requête 02.

Nombre de logements avec des disponibilités

```
> db.Paris.countDocuments({ has_availability: true })
```

Résultat : 90173

```
> db.Paris.countDocuments({ has_availability: true })
```

```
< 90173
```

R1. Nombre d'annonces par type de location

```
> db.Paris.aggregate([
  {
    $group: {
      _id : "$room_type",
      nb_annonces: { $sum: 1 }
    }
  }
])
< {
  _id: 'Private room',
  nb_annonces: 8975
}
{
  _id: 'Entire home/apt',
  nb_annonces: 85733
}
{
  _id: 'Shared room',
  nb_annonces: 401
}
{
  _id: 'Hotel room',
  nb_annonces: 776
}
```

R2. Les 5 annonces de location avec le plus d'évaluations avec leurs nombres d'évaluations

```
> db.Paris.aggregate([
  { $sort: { number_of_reviews: -1 } },
  { $limit: 5 },
  {
    $project: {
      annonce: "$name",
      nombre_avis: "$number_of_reviews",
      _id: 0
    }
  }
])
< {
  annonce: 'Sweet & cosy room next to Canal Saint Martin ❤️',
  nombre_avis: 3067
}
{
  annonce: 'Double/Twin Room, close to Opera and the Louvre with breakfast included',
  nombre_avis: 2620
}
{
  annonce: 'Bed in Dorm of 8 Beds "The Big One" in Paris',
  nombre_avis: 2294
}
{
  annonce: 'Comfortable bed in shared rooms of 8 in Paris 12e',
  nombre_avis: 2105
}
{
  annonce: 'Nice Room for 2 people',
  nombre_avis: 2048
}
```

R3. nombre total d'hôtes différents

```
> db.Paris.distinct("host_id").length
< 71979
```

R4. Nombre de locations réservables instantanément et sa proportion

```
> db.Paris.aggregate([
  {
    $group: {
      _id: null,
      total_annonces: { $sum: 1 },
      instant_bookable: {
        $sum: { $cond: [{ $eq: ["$instant_bookable", true] }, 1, 0] }
      }
    }
  },
  {
    $project: {
      _id: 0,
      total_annonces: 1,
      instant_bookable: 1,
      proportion_en_pourcentage: {
        $multiply: [ { $divide: ["$instant_bookable", "$total_annonces"] }, 100 ]
      }
    }
  }
])
< {
  total_annonces: 95885,
  instant_bookable: 22094,
  proportion_en_pourcentage: 23.042185951921574
}
```


R5. Hôtes avec plus de 100 annonces : 113La proportion sur le total des hôtes : **0.157%**

```
> db.Paris.aggregate([
  {
    $group: {
      _id: "$host_id",
      nb_annonces: { $max: "$host_total_listings_count" }
    }
  },
  {
    $group: {
      _id: null,
      total_hotes: { $sum: 1 },
      hotes_100_plus: {
        $sum: { $cond: [{ $gt: ["$nb_annonces", 100] }, 1, 0] }
      }
    }
  },
  {
    $project: {
      _id: 0,
      total_hotes: 1,
      hotes_100_plus: 1,
      pourcentage: {
        $multiply: [ { $divide: ["$hotes_100_plus", "$total_hotes"] }, 100 ]
      }
    }
  }
])
< {
  total_hotes: 71979,
  hotes_100_plus: 113,
  pourcentage: 0.15699023326247932
}
```

R6. Super hôtes différents : 10027La proportion sur le total des hôtes : **13.93%**

```
> db.Paris.aggregate([
  {
    $facet: {
      superhosts: [
        { $match: { host_is_superhost: true } },
        { $group: { _id: "$host_id" } },
        { $count: "nombre" }
      ],
      total_hotes: [
        { $group: { _id: "$host_id" } },
        { $count: "total" }
      ]
    }
  },
  {
    $project: {
      nombre_superhosts: { $arrayElemAt: ["$superhosts.nombre", 0] },
      total_hotes: { $arrayElemAt: ["$total_hotes.total", 0] },
      pourcentage: {
        $multiply: [
          { $divide: [ { $arrayElemAt: ["$superhosts.nombre", 0] }, { $arrayElemAt: ["$total_hotes.total", 0] } ] },
          100
        ]
      }
    }
  }
])
< {
  nombre_superhosts: 10027,
  total_hotes: 71979,
  pourcentage: 13.930451937370622
}
```

Chargement des données

- importation des librairies pymongo et polars
- connexion à MongoDB
- charger la bdd et la collection
- extraction des champs utiles
- conversion en dataframe

```
import pymongo
import polars as pl

# === 1. Connexion à MongoDB ===
client = pymongo.MongoClient("mongodb://localhost:27017/")
db = client["P7"]
collection = db["Paris"]

# === 2. Extraction des champs utiles (plus rapide que tout charger) ===
projection = {
    "room_type": 1,
    "availability_365": 1,
    "number_of_reviews": 1,
    "host_is_superhost": 1,
    "neighbourhood_cleansed": 1
}

data = list(collection.find({}, projection))

# Conversion en DataFrame Polars
df = pl.DataFrame(data)
```


Requête 1 : Calculer le taux de réservation moyen par mois par type de logement

```
taux_par_type = (  
    df.group_by("room_type")  
      .agg(pl.col("taux_reservation_mensuel").mean().alias("taux_moyen_mensuel"))  
      .with_columns(  
          | (pl.col("taux_moyen_mensuel") * 100).round(2).alias("taux_moyen_mensuel_%")  
      )  
)
```



1. Taux de réservation moyen par mois par type de logement
shape: (4, 2)

room_type	taux_moyen_mensuel_%
---	---
str	f64
Shared room	5.28
Entire home/apt	5.41
Hotel room	4.36
Private room	5.73

Requête 2 : Calculer la médiane des nombre d'avis pour tous les logements

```
mediane_global = df["number_of_reviews"].median()
```



2. Médiane du nombre d'avis (tous logements) : 3.0

Requête 3 : Calculer la médiane des nombre d'avis par catégorie d'hôte

```
mediane_par_categorie = (  
    df.group_by("categorie_hote")  
      .agg(pl.col("number_of_reviews").median().alias("mediane_avis"))  
      .sort("mediane_avis", descending=True)  
)
```



3. Médiane du nombre d'avis par catégorie d'hôte
shape: (3, 2)

categorie_hote	mediane_avis
---	---
str	f64
Superhost	24.0
Hôte non classé	12.5
Hôte classique	2.0

Requête 4 : Calculer la densité de logements par quartier de Paris

```
densite_quartier = (
    df.group_by("neighbourhood_cleansed")
      .agg(pl.len().alias("nombre_logements"))
      .sort("nombre_logements", descending=True)
)
```



4. Densité de logements par quartier (top 20)
shape: (20, 2)

neighbourhood_cleansed	nombre_logements
---	---
str	u32
Buttes-Montmartre	10555
Popincourt	8430
Vaugirard	7802
Batignolles-Monceau	6857
Entrepôt	6558
--	
Élysée	2898
Hôtel-de-Ville	2821
Palais-Bourbon	2740
Luxembourg	2701
Louvre	2026

Requête 5 : Identifier les quartiers avec le plus fort taux de réservation par mois

```
taux_quartier = (
    df.group_by("neighbourhood_cleansed")
      .agg(pl.col("taux_reservation_mensuel").mean().alias("taux_moyen_mensuel"))
      .sort("taux_moyen_mensuel", descending=True)
      .with_columns(
        (pl.col("taux_moyen_mensuel") * 100).round(2).alias("taux_moyen_mensuel_%")
      )
      .head(10) # Top 10
)
```



5. Top 10 quartiers avec le plus fort taux de réservation mensuel
shape: (10, 2)

neighbourhood_cleansed	taux_moyen_mensuel_%
---	---
str	f64
Ménilmontant	5.92
Buttes-Chaumont	5.81
Buttes-Montmartre	5.77
Entrepôt	5.75
Popincourt	5.74
Gobelins	5.69
Reuilly	5.65
Panthéon	5.53
Vaugirard	5.49
Observatoire	5.43

Prérequis

Power BI Desktop : version gratuite

Power BI Connector 2.14 : fait le pont entre MongoDB et Power BI

MongoDB ODBC 1.4.7 : driver permettant de traduire des requêtes SQL en requêtes MongoDB

Etape 1

Après avoir installé Power BI Connector, lancer l'exécutable **mongosqld.exe*** pour connaître le port de mongoDB : **127.0.0.1:3307 (localhost:3307)**

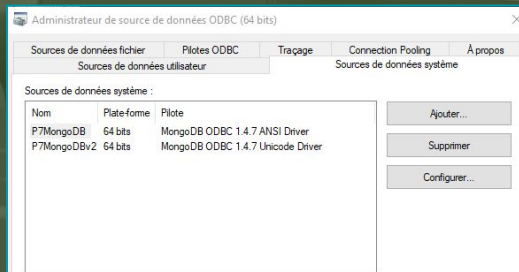
```
Version: 61e02a37c9efa342c768507171113ca6a1766616
Version OpenSSL 3.6.0 1 Oct 2025 (built with OpenSSL 3.6.0)
{ }
WARNING: Access control is not enabled for mongosqld.
g for connections at 127.0.0.1:3307
```

*Emplacement :

C:\Program Files\MongoDB\Connector for BI\2.14\bin

Etape 2

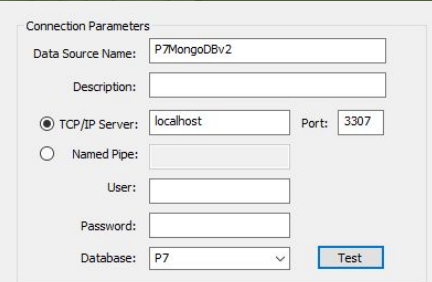
Après avoir installé MongoDB ODBC, lancer **Sources de données ODBC (64 bits)***



*Trouvable avec la recherche Windows

Etape 3

Ajouter une nouvelle source de données système avec les informations suivantes :



Le choix de la Database est possible qu'après un test de connexion (via le bouton)

Dans Power BI Desktop

- cliquer sur Obtenir les données
- choisir ODBC
- charger le/les collections

Obtenir les données

odbc

Tout

Base de données

Autre

Tout

ODBC

Kerberos ODBC

Importez des données à partir de ODBC.

À partir de ODBC

Nom de source de données

P7MongoDB

P7MongoDB

P7MongoDBv2

(Aucun)

OK

Annuler

Navigateur

Options d'affichage

ODBC (dsn=P7MongoDBv2) [3]

INFORMATION_SCHEMA

mysql

P7 [3]

Paris

Paris_amenities

Paris_host_verifications

Paris

_id	accommodates	availability_30	availability_365
695eec640b4d975cdd77cc61	3	0	
695eec640b4d975cdd77cc62	2	29	34
695eec640b4d975cdd77cc63	1	11	14
695eec640b4d975cdd77cc64	2	13	15
695eec640b4d975cdd77cc65	4	9	27
695eec640b4d975cdd77cc66	1	16	14

Les données dans l'aperçu ont été tronquées en raison de limites de taille.

Sélectionner les tables associées

Charger

Transformer les données

Annuler

Sans titre - Power BI Desktop

Se connecter

Fichier Accueil Insérer Modélisation Afficher Optimiser Aide

Partager

À propos de Infos

Formation guidée

Vidéos de formation

Documentation

Support

Aide

Blog Power BI

Communauté

Power BI pour les développeurs

Communauté

Soumettre une idée

Outils externes

Exemples de services de conseil

Ressources

Vos requêtes comprennent des modifications en attente qui n'ont pas été appliquées.

Appliquer les modifications

Ignorer les changements

Générer des visuels avec vos données

Sélectionnez ou faites glisser les champs du volet Données sur le canevas de rapport.

Visualisations

Générer un élément visuel

Données

Rechercher

Paris (3)

- ☐ _id
- ☐ Σ accommodates
- ☐ Σ availability_30
- ☐ Σ availability_365
- ☐ Σ availability_60
- ☐ Σ availability_90
- ☐ Σ bathrooms
- ☐ bathrooms_text
- ☐ Σ bedrooms
- ☐ Σ beds
- ☐ Σ calculated_ho...
- ☐ Σ calculated_ho...
- ☐ Σ calculated_ho...
- ☐ Σ calculated_ho...

Valeurs

Ajouter des champs de don...

Extraire

Page 1

50 %

03 | Concevez votre base de données

Répliquer les données

```
# Exemple pour le premier replica  
mongod --replSet rs0 --port 27017 --dbpath "C:\\...\\data\\rs1" --bind_ip localhost
```

URI ⓘ

Edit Connection String ⓘ

```
mongodb://localhost:27017,localhost:27018,localhost:27019/?replicaSet=rs0
```

Création des dossiers
pour chaque réplique

Lancer les instances
MongoDB sur
différents terminaux

Initialiser le replica
set sur un autre
terminal

Connexion dans Compass

exemple pour 3 répliques :

- C:\\...\\data\\rs1
- C:\\...\\data\\rs2
- C:\\...\\data\\rs3

```
mongosh --port 27017  
  
rs.initiate({  
  _id: "rs0",  
  members: [  
    { _id: 0, host: "localhost:27017" }, // primaire  
    { _id: 1, host: "localhost:27018" }, // secondaire 1  
    { _id: 2, host: "localhost:27019" } // secondaire 2  
  ]  
})
```


03 | Concevez votre base de données

Test des répliques

```
PS C:\Users\Phana\Desktop\Formation - Data Engineer\Projet\07 - Concevez et analysez une base de données NoSQL> mongosh --port 27017
Current Mongosh Log ID: 6962913714caa404431e2620
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:  8.2.3
Using Mongosh:  2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-01-10T18:37:06.802+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

rs0 [direct: primary] test> use P7
switched to db P7
rs0 [direct: primary] P7> show collections
Annonces
rs0 [direct: primary] P7> db.createCollection("testreplica")
{ ok: 1 }
rs0 [direct: primary] P7> exit
PS C:\Users\Phana\Desktop\Formation - Data Engineer\Projet\07 - Concevez et analysez une base de données NoSQL> mongosh --port 27018
Current Mongosh Log ID: 696291b2e6ed285a071e2620
Connecting to:  mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:  8.2.3
Using Mongosh:  2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-01-10T18:35:39.138+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

rs0 [direct: secondary] test> use P7
switched to db P7
rs0 [direct: secondary] P7> show collections
Annonces
testreplica
rs0 [direct: secondary] P7> |
```

03 | Concevez votre base de données

Distribuer les données en sharding

Stratégie de distribution du cluster

Nom	Router mongos (terminaux différents)	Port utilisé	Initialisation du replica
cfgRS (Config Servers)	<code>mongod --configsvr --replSet cfgRS --port 27020 --dbpath "data\configsvr" --bind_ip localhost</code>	<code>mongosh --port 27020</code>	<code>rs.initiate({ _id: "cfgRS", configsvr: true, members: [{ _id: 0, host: "localhost:27020" }]})</code>
shardParis (Shards)	<code>mongod --shardsvr --replSet shardParis --port 27021 --dbpath "data\shardParis" --bind_ip localhost</code>	<code>mongosh --port 27021</code>	<code>rs.initiate({ _id: "shardParis", members: [{ _id: 0, host: "localhost:27021" }]})</code>
shardLyon (Shards)	<code>mongod --shardsvr --replSet shardLyon --port 27022 --dbpath "data\shardLyon" --bind_ip localhost</code>	<code>mongosh --port 27022</code>	<code>rs.initiate({ _id: "shardLyon", members: [{ _id: 0, host: "localhost:27022" }]})</code>

définition

Config Servers Stockent les métadonnées du cluster

Shards Stockent les données réelles

mongos Routeur qui dirige les requêtes vers le bon shard

03 | Concevez votre base de données

Distribuer les données en sharding

Ouverture d'un routeur mongos pour recevoir les requêtes clients et les rediriger vers les bons shards sur un nouveau port : 27023

```
mongos --configdb cfgRS/localhost:27020 --port 27023 --bind_ip localhost
```

Export/Import des données

```
# EXPORT
mongoexport --uri="mongodb://localhost:27017,localhost:27018,localhost:27019/?replicaSet=rs0" --db=P7 --collection=Annonces --out=Annonces.json

# IMPORT
mongoimport --uri="mongodb://localhost:27023/P7" --collection=Annonces --file=Annonces.json
```

Configuration des shards
Paris/Lyon

```
mongosh --port 27023

# ajoute un shard au cluster
sh.addShard("shardParis/localhost:27021")
sh.addShard("shardLyon/localhost:27022")

# autorise le sharding pour la base de données P7
sh.enableSharding("P7")

# active le sharding sur la collection Annonces
# chaque document est placé sur un shard selon la valeur de host_location
sh.shardCollection("P7.Annonces", { host_location: 1 })

# création de tags associés aux shards
sh.addShardTag("shardParis", "PARIS")
sh.addShardTag("shardLyon", "LYON")

# définit une zone de valeurs du shard key (host_location)
sh.addTagRange( "P7.Annonces", { host_location: "Paris, France" }, { host_location: "Paris, France" }, "PARIS" )
sh.addTagRange( "P7.Annonces", { host_location: "Lyon, France" }, { host_location: "Lyon, France" }, "LYON" )
```


03 | Concevez votre base de données

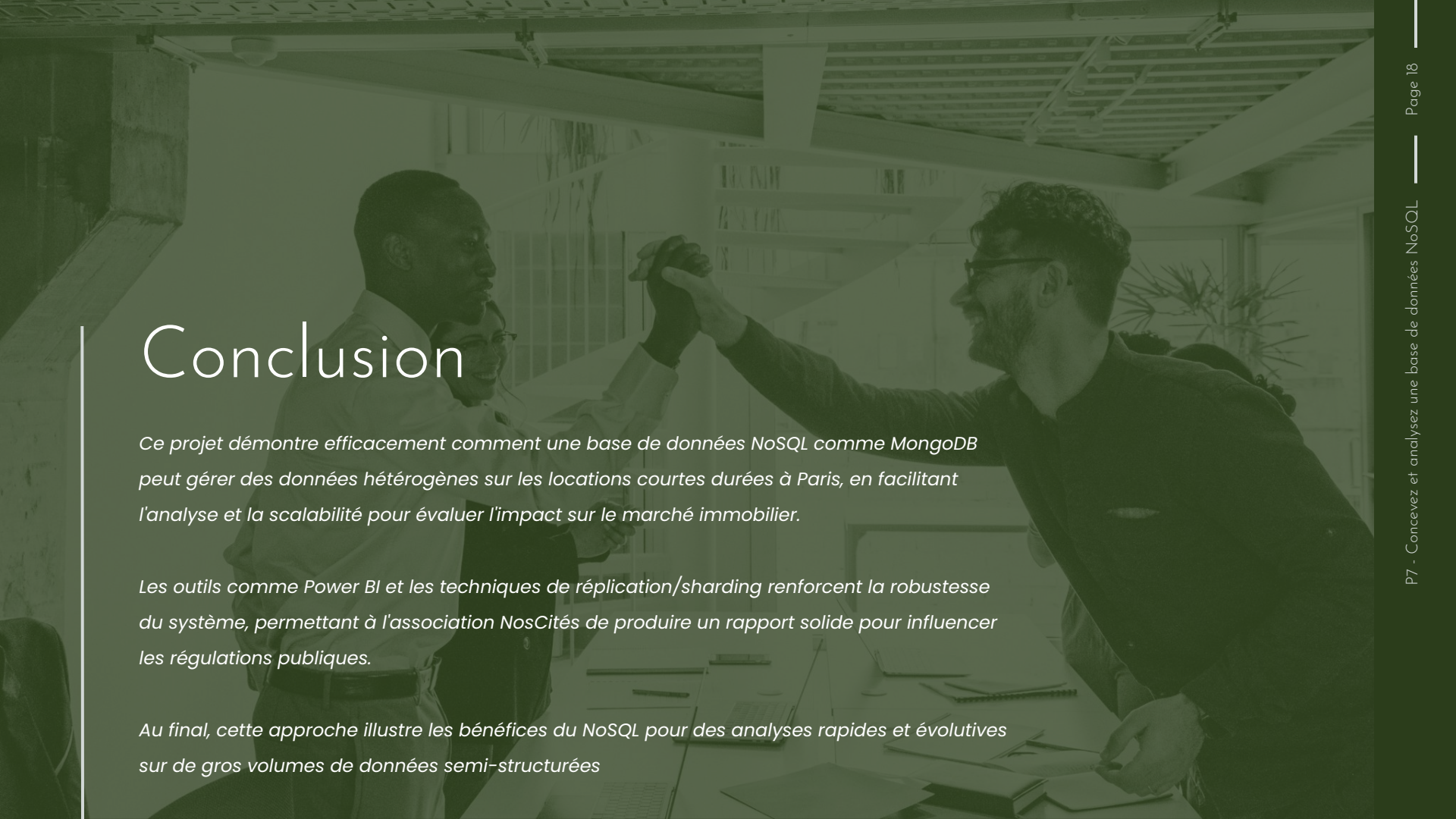
Test du cluster shardé

Répartition des données

```
[direct: mongos] test> use P7
switched to db P7
[direct: mongos] P7> db.Annonces.getShardDistribution()
Shard shardParis at shardParis/localhost:27021
{
  data: '219.65MiB',
  docs: 66262,
  chunks: 1,
  'estimated data per chunk': '219.65MiB',
  'estimated docs per chunk': 66262
}
---
Shard shardLyon at shardLyon/localhost:27022
{
  data: '128.52MiB',
  docs: 105858,
  chunks: 4,
  'estimated data per chunk': '32.13MiB',
  'estimated docs per chunk': 26464
}
---
Totals
{
  data: '348.17MiB',
  docs: 172120,
  chunks: 5,
  'Shard shardParis': [
    '63.08 % data',
    '38.49 % docs in cluster',
    '3KiB avg obj size on shard'
  ],
  'Shard shardLyon': [
    '36.91 % data',
    '61.5 % docs in cluster',
    '3KiB avg obj size on shard'
  ]
}
```

diagnostic la plus complète du sharding MongoDB

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
database: {
  _id: 'P7',
  primary: 'shardParis',
  version: {
    uuid: UUID('3cc66134-52d1-40c0-a4b8-baa6edb51d59'),
    timestamp: Timestamp({ t: 1768179191, i: 10 }),
    lastMod: 1
  }
},
collections: {
  'P7.Annonces': {
    shardKey: { host_location: 1 },
    unique: false,
    balancing: true,
    allowMigrations: true,
    chunkMetadata: [
      { shard: 'shardLyon', nChunks: 4 },
      { shard: 'shardParis', nChunks: 1 }
    ],
    chunks: [
      { min: { host_location: MinKey() }, max: { host_location: 'Lyon, France' }, 'on shard': 'shardLyon', 'last modified': Timestamp({ t: 2, i: 1 }) },
      { min: { host_location: 'Lyon, France' }, max: { host_location: 'Lyon, France@' }, 'on shard': 'shardLyon', 'last modified': Timestamp({ t: 1, i: 2 }) },
      { min: { host_location: 'Lyon, France@' }, max: { host_location: 'Paris, France' }, 'on shard': 'shardLyon', 'last modified': Timestamp({ t: 1, i: 4 }) },
      { min: { host_location: 'Paris, France' }, max: { host_location: 'Paris, France@' }, 'on shard': 'shardParis', 'last modified': Timestamp({ t: 2, i: 0 }) },
      { min: { host_location: 'Paris, France@' }, max: { host_location: MaxKey() }, 'on shard': 'shardLyon', 'last modified': Timestamp({ t: 1, i: 6 }) }
    ],
    tags: [
      {
        tag: 'LYON',
        min: { host_location: 'Lyon, France' },
        max: { host_location: 'Lyon, France@' }
      },
      {
        tag: 'PARIS',
        min: { host_location: 'Paris, France' },
        max: { host_location: 'Paris, France@' }
      }
    ]
  }
}
```



Conclusion

Ce projet démontre efficacement comment une base de données NoSQL comme MongoDB peut gérer des données hétérogènes sur les locations courtes durées à Paris, en facilitant l'analyse et la scalabilité pour évaluer l'impact sur le marché immobilier.

Les outils comme Power BI et les techniques de répllication/sharding renforcent la robustesse du système, permettant à l'association NosCités de produire un rapport solide pour influencer les régulations publiques.

Au final, cette approche illustre les bénéfices du NoSQL pour des analyses rapides et évolutives sur de gros volumes de données semi-structurées